Modern Bayesian Workflow and Probabilistic Programming Languages

Eliezer de Souza da Silva

University of Coimbra, DEI, CISUC
https://sereliezer.github.io/
eliezer.silva@uc.pt
Probabilistic AI School (Vilnius)

Day 1 afternoon tutorial — August 3, 2026

What is this session about

Morning: *how to specify probabilistic models and mathematical foundations for inference?*

Afternoon: **how do we make it executable?**

Same ideas, a computational lens

Morning: probabilistic model	**Afternoon: probabilistic program**
Joint factorization	Execution trace with named sample sites
Conditioning on evidence	Observations attached to sample sites
Repeated independent observations	A `plate` and tensor shapes
Posterior inference	An inference algorithm over program traces
Build–compute–critique–repeat	Generate–infer–check–revise in code

We begin where the morning ends

- a fully probabilistic linear-regression model in plate notation;
- prior and posterior predictive distributions;
- exact inference versus Monte Carlo and variational approximations;
- the probabilistic modeling cycle: build, compute, critique, repeat.

Now we turn each of these into an executable interface using Pyro (with some mentions to NumPyro, PyMC, Stan and others)

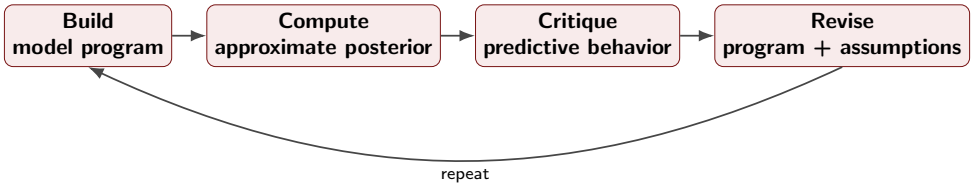
Plan

Part	Block	Mode
A	Probabilistic models as executable programs	Slides + discussion
A	Pyro model, guides, ELBO, and SVI	Slides + Notebook to-run
A	Examples: Regression + Temporal Models	Notebook to-run
B	Exercises: Extending Regression + Temporal Models	Notebook to-do
C	Modern workflow: Amortized Inference	Slides
C	Modern workflow: LLMs and Agentic Systems	Slides

Plan

We will work on three notebooks (link in Github), two of them are code to be run and analyzed, and one of them is code to be completed as an exercize.

The modeling cycle becomes executable



Source / pointer: D. Blei (2014), "Build, compute, critique, repeat."

A program connects syntax to computation

Syntax

- a finite expression in a programming language;
- built according to the language's grammar.

Semantics

- assigns computational meaning to that expression;
- an interpreter or compiler realizes that meaning on a machine.

$$\text{program text} \xrightarrow{\text{language semantics}} \text{computation}$$

A probabilistic model has a natural forward execution semantics

```
def model():
    z = pyro.sample(
        "z", dist.Normal(0., 1.)
    )
    x = pyro.sample(
        "x", dist.Normal(z, 0.5)
    )
    return x
```

Each execution:

1. instantiates distributions;
2. makes random choices;
3. returns a value and records a trace.

Repeated execution defines the joint distribution

$$p(x, z) = p(z)\, p(x \mid z).$$

Sampling primitives give a probabilistic program its generative semantics.

A PPL gives the model operational semantics

Mathematical model

$$p(x, z) = \prod_j p(z_j \mid \mathrm{pa}(z_j)) \prod_i p(x_i \mid \mathrm{pa}(x_i)).$$

It states how the joint distribution factorizes.

Probabilistic program

- runs the generative process;
- names random choices;
- records a trace;
- can be replayed, conditioned, transformed, and scored.

Target computation for a probabilistic program

The same joint model $p(x, z)$ supports several computations:

Question	**Target computation**
What data could the model generate?	Draw $(z, x) \sim p(z, x)$
What is plausible after observing x?	Draw from or summarize $p(z \mid x)$
How probable is an event A?	Compute $\Pr(z \in A \mid x)$
How well does the model predict x?	Compute or estimate $p(x)$

The model says what is random and how variables relate; the query says what we want computed.

Two computational tasks recur across probabilistic inference

Sampling from a target

$$z^{(1)}, \ldots, z^{(S)} \sim \pi(z).$$

Use draws to explore uncertainty, form predictions, and approximate expectations:

$$\mathbb{E}_{\pi}[h(z)] \approx \frac{1}{S} \sum_{s=1}^{S} h(z^{(s)}).$$

Marginalization

$$p(x) = \int p(x, z) \, \mathrm{d}z.$$

Integrate out latent quantities to obtain probabilities, normalizing constants, predictions, or summaries.

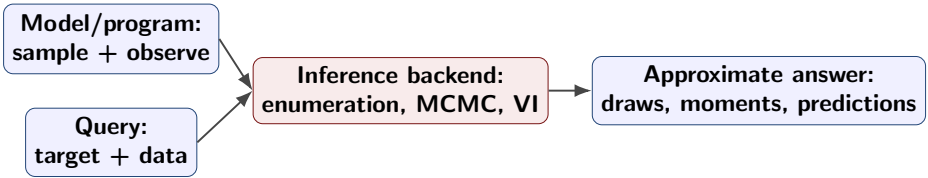
The hard integral sits at the center of probabilistic computations

For observed x, Bayesian inference targets

$$p(z \mid x) = \frac{p(x, z)}{p(x)} = \frac{p(x, z)}{\int p(x, z') \, \mathrm{d}z'}.$$

- Exact marginalization is usually unavailable in realistic models.
- MCMC targets $p(z \mid x)$ without explicitly evaluating $p(x)$, but must still explore where its probability mass lies.
- Variational inference replaces direct integration with optimization, usually using Monte Carlo estimates inside the objective.

A PPL turns model descriptions into inference interfaces



A probabilistic programming language provides reusable machinery and automate different part of the inference process. Automation of this process varies accross PPLs.

Monty Hall, now as an executable model

```
def monty_model(choice, opened=None):
    prize = pyro.sample("R", dist.Categorical(torch.ones(3) / 3))
    probs = host_policy(choice, prize) # p(H | C, R)
    pyro.sample("H", dist.Categorical(probs), obs=opened)
    return prize
```

- The morning's DAG $C \to H \leftarrow R$ is now a generator.
- `obs=opened` conditions the trace on Monty's action.
- Inference targets the unobserved site `R`.

Inference. We can use discrete enumeration which is a natural backend for this model.

Inference strategies

Backend	**Computational idea**	**Guideline**
Enumeration	Sum over discrete states	Small space
MCMC	Construct a posterior Markov chain	Mixing of chains
Variational inference	Optimize a tractable distribution	Adequate approximation family
Amortized inference	Learn an inference map across tasks	In-distribution or out-of-distribution

PPLs separate the **model interface** from the **inference strategy**, yet adequate choices must be made and they depend on the modeling choices.

Predictive checks test the program's implications

Prior predictive

$$\theta \sim \pi(\theta), \qquad y^{\text{rep}} \sim f(y \mid \theta).$$

Could the program generate plausible datasets a priori?

Posterior predictive

$$\theta \sim \pi(\theta \mid \mathcal{D}), \qquad y^{\text{rep}} \sim f(y \mid \theta).$$

Does the fitted program reproduce samples that follows the data?

A predictive check criticizes the full generative story and allows for model adjustment.

A Pyro model is a stochastic Python function

```
import torch
import pyro
import pyro.distributions as dist

def model(x, y=None):
    a = pyro.sample("a", dist.Normal(0.0, 5.0))
    b = pyro.sample("b", dist.Normal(0.0, 2.0))
    sigma = pyro.sample(
        "sigma", dist.LogNormal(-1.0, 0.5)
    )

    mu = a + b * x

    with pyro.plate("data", x.size(0)):
        pyro.sample(
            "y", dist.Normal(mu, sigma), obs=y
        )
```

A Pyro model is a stochastic Python function

- Ordinary Python computes the deterministic mean $\mu = a + bx$.
- `pyro.sample` introduces named random variables.
- `obs=y` turns a random choice into observed evidence.
- Running with `y=None` generates data from the model.

Sample statements define the model's execution trace

Each call to `pyro.sample(name, distribution)` creates a sample site.

Site	Observed?	Distribution	Role
`a`	No	$\mathcal{N}(0, 5)$	Intercept
`b`	No	$\mathcal{N}(0, 2)$	Slope
`sigma`	No	LogNormal$(-1, 0.5)$	Noise scale
`y`	If supplied	$\mathcal{N}(a + bx, \sigma)$	Observations

A site records:

- its unique name and sampled or observed value;
- its distribution and tensor shape;
- whether it is observed;
- its contribution to the joint log density.

Effect handlers can inspect and transform execution

```
import pyro.poutine as poutine

conditioned_model = poutine.condition(
    model, data={"y": y}
)

trace = poutine.trace(conditioned_model).get_trace(x)
trace.compute_log_prob()

for name, node in trace.nodes.items():
    if node["type"] == "sample":
        print(
            name,
            node["value"].shape,
            node["log_prob"].sum()
        )
```

Effect handlers can inspect and transform execution

- `trace` records the random choices made during execution.
- `condition` supplies values to named sample sites.
- `replay` reruns a model using choices from another trace.
- Inference algorithms are compositions of these transformations.

Plates and events express different structures

```
def vector_regression(X, y=None):
    D = X.size(-1)
    N = X.size(0)

    w = pyro.sample(
        "w",
        dist.Normal(
            torch.zeros(D), torch.ones(D)
        ).to_event(1)
    )
    sigma = pyro.sample(
        "sigma", dist.LogNormal(-1.0, 0.5)
    )

    with pyro.plate("data", N):
        pyro.sample(
            "y",
            dist.Normal(X @ w, sigma),
            obs=y
        )
```

Plates and events express different structures

- `to_event(1)` treats the D coordinates of $\mathbf{w}$ as one vector-valued event.
- `plate("data", N)` declares N conditionally independent observations.
- Event dimensions are summed together; plate dimensions may be vectorized or subsampled.

Many Pyro errors are shape errors that reveal an unclear independence assumption.

Plates and events encode different structure

`to_event(1)`

- treats the two coordinates of **w** as one vector-valued event;
- log probabilities sum over that event dimension;
- makes no claim that observations repeat.

`plate("data", N)`

- declares conditional independence across observations;
- enables minibatching and vectorized inference;
- corresponds to the plate in the morning graphical model.

Shape errors are often modeling errors in disguise. Always reason about batch shape, event shape, and plate dimensions together.

Source / pointer: Pyro documentation: tensor shapes and `pyro.plate`.

Learnable quantities live in the parameter store

```
from torch.distributions import constraints

a_loc = pyro.param(
    "a_loc", torch.tensor(0.0)
)

a_scale = pyro.param(
    "a_scale",
    torch.tensor(0.2),
    constraint=constraints.positive
)

pyro.clear_param_store()
```

Learnable quantities live in the parameter store

- `pyro.param` registers a persistent learnable tensor.
- Reusing a parameter name retrieves the same tensor.
- Constraints map unconstrained optimizer variables onto valid supports.
- `clear_param_store()` prevents state from leaking between fits.

Sample sites describe random variables of the model; parameters describe quantities optimized by the inference procedure.

A guide is an executable posterior approximation

The guide must reproduce every unobserved model site with the same name and compatible support.

The model defines the joint density

$$p_{\mathcal{M}}(\mathcal{D}, \theta) = \pi(\theta) f(\mathcal{D} \mid \theta),$$

while the guide defines a tractable distribution

$$q_\phi(\theta \mid \mathcal{D}) \approx \pi(\theta \mid \mathcal{D}).$$

In Pyro, both are programs. They must agree on:

- names of latent sample sites;
- support and event dimensions;
- conditioning inputs and plate structure.

VI turns conditioning into optimization

Start with the evidence:

$$\log p(\mathcal{D}) = \underbrace{\mathbb{E}_{q_\phi}[\log p_{\mathcal{M}}(\mathcal{D}, \theta) - \log q_\phi(\theta)]}_{\mathcal{L}(\phi)\ \text{(ELBO)}} + \underbrace{\mathrm{KL}(q_\phi(\theta) \,\|\, p(\theta \mid \mathcal{D}))}_{\geq 0}.$$

Therefore maximizing $\mathcal{L}(\phi)$ minimizes the reverse KL divergence.

- Monte Carlo estimates the expectation.
- Automatic differentiation supplies gradients.
- An optimizer updates the guide parameters ϕ.

What must a guide decide?

Decision	Example consequence
Factorization	Mean-field $q(\mathbf{w})q(\sigma)$ cannot represent dependence.
Marginal family	Gaussian factors may miss skewness or multiple modes.
Support transform	A positive σ needs a transformation or positive-support family.
Initialization	Poor initial scale can destabilize or slow optimization.

Changing the guide changes what posterior shapes the algorithm is allowed to find.

A mean-field guide

```
def mean_field_guide(x, y=None):
    w_loc = pyro.param("w_loc", torch.zeros(2))
    w_scale = pyro.param(
        "w_scale", 0.1 * torch.ones(2), constraint=constraints.positive
    )
    sigma_loc = pyro.param("sigma_loc", torch.tensor(0.0))
    sigma_scale = pyro.param(
        "sigma_scale", torch.tensor(0.1), constraint=constraints.positive
    )

    pyro.sample("w", dist.Normal(w_loc, w_scale).to_event(1))
    pyro.sample("sigma", dist.LogNormal(sigma_loc, sigma_scale))
```

- `pyro.param` registers learnable variational parameters.
- Constraints keep scales positive during optimization.
- Site names `w` and `sigma` match the model exactly.

Automatic guides

```
from pyro.infer.autoguide import (
    AutoNormal,
    AutoMultivariateNormal,
    AutoLowRankMultivariateNormal,
)

mean_field = AutoNormal(model)
full_rank = AutoMultivariateNormal(model)
low_rank = AutoLowRankMultivariateNormal(
    model, rank=2
)
```

What AutoNormal automates

`AutoNormal(model)` constructs a mean-field guide automatically:

1. inspect each continuous latent sample site;
2. map constrained values to an unconstrained space;
3. place an independent Normal variational factor there;
4. learn a location and scale;
5. transform samples back to the site's support.

Source / pointer: Pyro `infer.autoguide.AutoNormal` documentation.

Automatic guides classes

Guide	Can represent	Main trade-off
`AutoNormal`	Marginal scales	No posterior correlation
`AutoMultivariateNormal`	Dense correlation	Quadratic parameter cost
`AutoLowRank...`	Low-rank correlation	Rank must be chosen

Automatic guides:

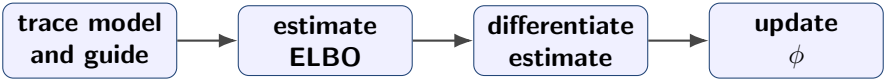
- discover latent sites by tracing the model;
- transform constrained variables to unconstrained space;
- create and manage variational parameters.

SVI connects model, guide, loss, and optimizer

```
from pyro.infer import SVI, Trace_ELBO
from pyro.optim import Adam

pyro.clear_param_store()

svi = SVI(
    model=model,
    guide=guide,
    optim=Adam({"lr": 0.02}),
    loss=Trace_ELBO()
)

losses = []
for step in range(2000):
    loss = svi.step(x, y)
    losses.append(loss)
```

SVI connects model, guide, loss, and optimizer

```
pyro.clear_param_store()
guide = AutoNormal(regression_model)
optimizer = pyro.optim.Adam({"lr": 0.03})
svi = SVI(regression_model, guide, optimizer, loss=Trace_ELBO())

losses = []
for step in range(1200):
    loss = svi.step(x, y) # sample, score, differentiate, update
    losses.append(loss / len(x))
```



SVI connects model, guide, loss, and optimizer

Each call to `svi.step(x, y)`:

1. executes the guide and model;
2. estimates the negative ELBO;
3. differentiates through the sampled computation;
4. updates the guide parameters.

Predictive turns fitted inference back into simulation

```
from pyro.infer import Predictive
predictive = Predictive(
    model,
    guide=guide,
    num_samples=1000,
    return_sites=("a", "b", "sigma", "y")
)
samples = predictive(x, y=None)
print(samples["a"].shape) # [1000]
print(samples["y"].shape) # [1000, N]
```

The returned samples support different questions:

- a, b, and sigma: what has inference learned?
- y: what datasets does the fitted model predict?
- intervals and discrepancies: where does predictive behavior fail?

The complete Pyro workflow

1. **Simulate**: run the model without observations.
2. **Condition**: attach evidence to named sample sites.
3. **Inspect**: trace sites, values, shapes, and log probabilities.
4. **Approximate**: choose a guide whose geometry matches the task.
5. **Optimize**: fit the guide with SVI and an ELBO estimator.
6. **Predict**: generate replicated datasets with `Predictive`.
7. **Criticize**: compare predictive implications with relevant discrepancies.
8. **Revise**: change one model or inference assumption at a time.

Code Along Notebooks

- Regression Github and Colab
- Dynamic models: Linear State Space/SIR GitHub and Colab

Bayesian Regression Notation

We fix the notation $\mathbf{x}_i = [1, x_i]^\top$ and

$$Y_i \mid \mathbf{w}, \mathbf{x}_i \sim \mathcal{N}(\mathbf{w}^\top \mathbf{x}_i, \gamma^{-1}), \qquad \mathbf{w} \sim \mathcal{N}(\mathbf{0}, I).$$

For implementation we write $\sigma = \gamma^{-1/2}$ and make its uncertainty explicit:

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, s_w^2 I), \qquad \sigma \sim \text{LogNormal}(m_\sigma, s_\sigma), \qquad Y_i \mid \mathbf{w}, \sigma, \mathbf{x}_i \sim \mathcal{N}(\mathbf{w}^\top \mathbf{x}_i, \sigma^2).$$

The data-generating is $y = 3 + 2x + \epsilon$.
Link to Github and Colab

Before inference: simulate the prior predictive

$$\mathbf{w} \sim \mathcal{N}(0, s_w^2 I), \quad \sigma \sim \text{LogNormal}(m_\sigma, s_\sigma), \quad y_i^{\text{rep}} \sim \mathcal{N}(\mathbf{w}^\top \mathbf{x}_i, \sigma^2).$$

Live questions:

- What slopes and intercepts do these priors imply on the observed scale?
- How often do we generate datasets we would immediately reject?
- If the simulation is absurd, can inference repair the model? *No.*

After fitting: inspect the approximation itself

The notebook makes the guide visible:

- print the learned locations and scales;
- draw samples from $q_\phi(\mathbf{w}, \sigma)$;

Posterior predictive checks target discrepancies

Rather than asking whether a plot "looks good," choose discrepancies $T(y)$:

- center and scale;
- tail quantiles and maximum residual;
- interval coverage;
- residual trend against x;
- heteroscedasticity across the predictor range.

$$T(y^{\mathrm{rep}}) \quad \text{versus} \quad T(y^{\mathrm{obs}}).$$

Revision on the likelihood

Replace

$$Y_i \mid \mathbf{w}, \sigma, \mathbf{x}_i \sim \mathcal{N}(\mathbf{w}^\top \mathbf{x}_i, \sigma^2)$$

with

$$Y_i \mid \mathbf{w}, \sigma, \nu, \mathbf{x}_i \sim t_\nu(\mathbf{w}^\top \mathbf{x}_i, \sigma).$$

The mean structure and priors remain fixed, so changes in predictive behavior have a clear interpretation.

A state-space model separates dynamics from measurement

The notebook begins with a linear Gaussian model:

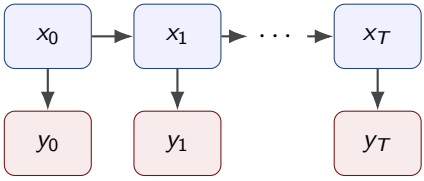
$$x_0 \sim \mathcal{N}(0, 2),$$

$$x_t \mid x_{t-1} \sim \mathcal{N}(\phi x_{t-1}, \sigma_x), \qquad t = 1, \ldots, T,$$

$$y_t \mid x_t \sim \mathcal{N}(x_t, \sigma_y).$$

Smoothing reconstructs the dependent latent path $x_{0:T}$.

Link to GitHub Colab



x_t: latent state, y_t: measurement

ϕ: persistence

σ_x: `process_scale`

σ_y: `obs_scale`

SIR embeds nonlinear dynamics inside the model

Let the epidemic state be

$$\mathbf{x}_t = (S_t, I_t, R_t), \qquad S_t + I_t + R_t = N.$$

For one discrete time step, define new infections and recoveries:

$$\Delta I_t^{\text{new}} = \beta \frac{S_t I_t}{N}, \qquad \Delta R_t = \gamma I_t,$$

$$S_{t+1} = S_t - \Delta I_t^{\text{new}}, \qquad I_{t+1} = I_t + \Delta I_t^{\text{new}} - \Delta R_t, \qquad R_{t+1} = R_t + \Delta R_t.$$

The observed case count is a noisy, partially reported view of incidence:

$$C_t \sim \text{Poisson}\left(\rho \, \Delta I_t^{\text{new}}\right).$$

β controls transmission, γ recovery, ρ reporting, and I_0 the initial infectious population.

The derived quantity $R_0 = \beta/\gamma$ helps explain the posterior dependence between β and γ.

The Pyro interface for dynamic models

Both examples have the same computational decomposition:

$$\theta \sim p(\theta), \qquad \mathbf{x}_t = f_\theta(\mathbf{x}_{t-1}, \varepsilon_t), \qquad y_t \sim p_\theta(y_t \mid \mathbf{x}_t).$$

	Linear Gaussian	**SIR**
State	scalar x_t	compartments (S_t, I_t, R_t)
Transition	stochastic and linear	nonlinear; deterministic given parameters
Observation	continuous Gaussian y_t	Poisson reported count C_t
Infer	$x_{0:T}, \phi, \sigma_x, \sigma_y$	β, γ, ρ, I_0
Guide	mean-field versus low-rank	full-rank over four parameters

In Pyro, time-indexed `sample` sites create the latent path; ordinary Python computes the deterministic SIR transition; and `obs=` attaches measurements.

Interval + Exercise

Notebook for code completing exercises GitHub Colab

Extra: NumPyro keeps the semantics and changes execution

	Pyro	**NumPyro**
Array system	PyTorch tensors	JAX arrays
Randomness	Global-style seeded state	Explicit PRNG keys
Parameters	Global parameter store	Functional parameter state
Compilation	Eager PyTorch by default	JAX transformations and JIT
Core language	`sample`, `plate`, handlers	Closely matching primitives

The NumPyro model is intentionally familiar

```
def numpyro_regression_model(x, y=None):
    w = numpyro.sample(
        "w", ndist.Normal(jnp.zeros(2), 2.5).to_event(1)
    )
    sigma = numpyro.sample("sigma", ndist.LogNormal(0.0, 0.5))
    mean = w[0] + w[1] * x

    with numpyro.plate("data", x.shape[0]):
        numpyro.sample("obs", ndist.Normal(mean, sigma), obs=y)
```

What changed? Imports and array types. What did not? Sample-site names, support, event dimensions, plate semantics, and the generative story.

NumPyro SVI makes state and randomness explicit

```
guide = AutoNormal(numpyro_regression_model)
svi = SVI(numpyro_regression_model, guide,
          Adam(0.03), loss=Trace_ELBO())

result = svi.run(jax.random.PRNGKey(0), 1200,
                 x_jax, y_jax, progress_bar=False)

predictive = Predictive(numpyro_regression_model,
    guide=guide, params=result.params, num_samples=500)
draws = predictive(jax.random.PRNGKey(1), x_jax, None)
```

Source / pointer: NumPyro documentation: Getting Started, SVI, AutoNormal, and Predictive.

Current and future of PPLs and Bayesian Workflow

We will introduced two important aspects of current developments:

- Amortized Inference: modern NN learning upfront a rich distribution that approximates the posterior (in different forms and ways)
- LLMs and agents in the Bayesian workflow.

A standard guide is fitted per dataset

For each dataset $\mathcal{D}_k$, classical VI solves a new optimization problem:

$$\phi_k^* = \arg\max_{\phi_k} \mathcal{L}(\phi_k; \mathcal{D}_k), \qquad q_{\phi_k}(\theta) \approx p(\theta \mid \mathcal{D}_k).$$

This is fast relative in some procedures, but the optimization cost is paid again for every dataset.

Amortization learns the map from data to a guide

An encoder with shared parameters ψ produces variational parameters:

$$h_\psi(\mathcal{D}) = (\mu_\psi(\mathcal{D}), s_\psi(\mathcal{D})), \qquad q_\psi(\theta \mid \mathcal{D}) = \mathcal{N}\big(\mu_\psi(\mathcal{D}), \operatorname{diag} s_\psi^2(\mathcal{D})\big)\,.$$

Prior-data simulation becomes a training distribution

$$\theta \sim \pi(\theta), \qquad \mathcal{D} \sim f(\mathcal{D} \mid \theta).$$

Pairs $(\mathcal{D}, \theta)$ train an inference operator that can be reused on new datasets. This pattern appears in:

- neural posterior estimation and simulation-based inference;
- prior-data fitted networks (e.g. TabPFN);
- repeated-task inference in hierarchical and latent-variable models.

Amortization changes some part of the workflow

Fast inference at deployment can hide substantial up-front choices:

- Which prior-data distribution generated training tasks?
- Which features summarize a variable-size dataset?
- Which posterior family is returned?
- How is calibration assessed under distribution shift?
- When should the system refuse or fall back to per-dataset inference?

The workflow moves to simulator design, training coverage, validation, and monitoring.

Generate–test–repair is not always Bayesian

An LLM can propose code, observe an error, and try again. That is an adaptive search loop. To make it properly Bayesian requires more:

- an explicit hypothesis space over models, programs, or workflows;
- a representation of uncertainty over those hypotheses;
- a principled update from predictive or execution evidence;
- decisions that account for uncertainty and utility.

Workflow expertise is becoming installable

Decision Hub packages skills for statistics, ML, and data science as reusable agent capabilities. The important abstraction is **workflow-as-skill**.

Source / pointer: Decision Hub, hub.decision.ai; PyMC Labs Decision Hub repository.

AutoStan closes a predictive model-revision loop

AutoStan is a 2026 research prototype in which a coding agent:

1. edits a Stan model;
2. runs MCMC;
3. evaluates held-out negative log predictive density;
4. checks divergences, $\widehat{R}$, and effective sample size;
5. keeps or reverts the change.

This is closer to Bayesian workflow because prediction and inference diagnostics guide model-space search. It still does not maintain a posterior distribution over model programs.

Source / pointer: O. Dürr (2026), *AutoStan: Autonomous Bayesian Model Improvement via Predictive Feedback*, arXiv:2603.27766.

Where Bayesian structure could enter

Let m denote a model program and r an implementation or inference recipe. A Bayesian system would target something like

$$p(m, r \mid \mathcal{D}, g, e) \propto p(\mathcal{D}, e \mid m, r, g)\, p(m, r \mid g),$$

where g is the modeling goal and e is execution or diagnostic evidence. Possible roles for an LLM:

- proposal kernel over structured programs;
- interpreter of diagnostics into candidate revisions;
- generator of tests and posterior predictive discrepancies.

Takeaways

1. A PPL gives probabilistic models operational semantics.
2. A guide is an executable approximation with consequential assumptions.
3. SVI/ELBO optimization is an inference procedure.
4. Agentic workflow becomes genuinely Bayesian only when it represents and updates uncertainty over alternatives.

Questions for the room

1. Which parts of model criticism should remain human-centered?
2. What should a posterior over programs mean operationally?
3. How should an agent expose uncertainty about its own model changes?

Closing provocation

Probabilistic programming made **models executable**.

Amortized inference made **inference reusable**.

Modeling agents may make **workflow executable**.

The open question is whether they will merely search—or maintain calibrated uncertainty over what to try, trust, and revise.

Notation crosswalk

Object	**Morning**	**Afternoon**
Data model	$f(x \mid \theta)$	$p(\mathcal{D} \mid \theta)$
Prior	$\pi(\theta)$	$\pi(\theta)$
Posterior	$\pi(\theta \mid x)$	$p(\theta \mid \mathcal{D})$
Regression weights	$\mathbf{w}$	$\mathtt{w} = (a, b)$
Noise	precision γ	scale $\sigma = \gamma^{-1/2}$
Approximation	variational approach	guide $q_\phi(\theta)$

Backup: ELBO derivation

$$
\begin{aligned}
\log p(\mathcal{D}) &= \mathbb{E}_q \left[\log p(\mathcal{D})\right] \\
&= \mathbb{E}_q \left[\log \frac{p(\mathcal{D}, \theta)}{p(\theta \mid \mathcal{D})}\right] \\
&= \mathbb{E}_q \left[\log \frac{p(\mathcal{D}, \theta)}{q(\theta)}\right] + \mathbb{E}_q \left[\log \frac{q(\theta)}{p(\theta \mid \mathcal{D})}\right] \\
&= \mathcal{L}(q) + \mathrm{KL}(q(\theta) \| p(\theta \mid \mathcal{D}))\,.
\end{aligned}
$$

Backup: common model–guide mismatches

- A latent site exists in the model but not the guide.
- A site name differs by one character.
- Event dimensions differ between model and guide.
- The guide proposes values outside the latent support.
- A plate dimension is missing or placed at a different tensor axis.
- Discrete latent variables are passed to a continuous reparameterized guide.

Backup: when mean field is not enough

Warning signs:

- strong correlations in MCMC reference fits;
- hierarchical funnels or weak identifiability;
- multimodality from symmetries or mixture labels;
- posterior predictive uncertainty systematically too narrow;
- sensitivity to initialization or optimizer settings.

Possible responses: reparameterize the model, use a richer guide, compare with MCMC on a reduced problem, or revise the model itself.

References and pointers I

- D. Blei (2014), "Build, compute, critique, repeat."
- Pyro documentation: https://docs.pyro.ai/
- Pyro SVI tutorials: https://pyro.ai/examples/svi_part_i.html
- NumPyro documentation: https://num.pyro.ai/
- NumPyro automatic guides: https://num.pyro.ai/en/stable/autoguide.html
- TabPFN: prior-data fitted networks for tabular prediction.

References and pointers II

- Decision Hub: https://hub.decision.ai/
- PyMC Labs (2025), "Write Me a PyMC Model."
- PyMC Labs (2026), "Improving AI Agent Performance with Domain-Specific Skills."
- PyMC Labs (2026), "The LLM Transpiler: 3–7x Faster PyMC Models with Rust."
- O. Dürr (2026), *AutoStan*, arXiv:2603.27766.
- Alchemize repository: https://github.com/pymc-labs/alchemize